Practical 2: MiniBank Customer Profile & Encapsulated Accounts

Problem Definition:

Create Customer and Account classes for MiniBank using encapsulation (private fields with public getters, no direct setter for balance), constructor chaining using this(), static counters for auto-generating unique IDs ('AC1001'), and safe deposit/withdraw transaction methods.

Code:

```
class PartB {
    public static class Customer {
        private String name;
        private String email;
        private String mobile;
        private final String customerId;
        private static long customerCounter = 100;

        private static String generateCustomerId() {
            customerCounter++;
            return "CUST" + customerCounter;
        }

        public Customer(String name, String email, String mobile) {
            this.name = name;
            this.email = email;
            this.mobile = mobile;
            this.customerId = generateCustomerId();
        }

        public String getName() { return name; }
        public String getEmail() { return email; }
        public String getMobile() { return mobile; }
        public String getCustomerId() { return customerId; }
    }

    public static class Account {
        private final String accountNumber;
        private String ownerName;
        private long balance;
        private boolean active;
        private static long accountCounter = 0;

        private static String generateAccountNumber() {
            accountCounter++;
            return "AC" + String.format("%04d", accountCounter);
        }

        public Account(String ownerName, long openingBalance) {
            this.ownerName = ownerName;
            this.accountNumber = generateAccountNumber();
            this.active = true;
            this.balance = (openingBalance >= 0) ? openingBalance : 0;
        }

        public Account(String ownerName) {
            this(ownerName, 0);
```

```
        }

        public void deposit(long amount) {
            if (amount <= 0) {
                System.out.println("Deposit failed! Amount must be positive.");
                return;
            }
            balance += amount;
            System.out.println("Deposited Rs. " + amount + " to " + accountNumber);
        }

        public boolean withdraw(long amount) {
            if (amount <= 0) {
                System.out.println("Withdrawal failed! Amount must be positive.");
                return false;
            }
            if (balance >= amount) {
                balance -= amount;
                System.out.println("Withdrew Rs. " + amount + " from " + accountNumber);
                return true;
            } else {
                System.out.println("Withdrawal failed! Insufficient funds.");
                return false;
            }
        }

        public static boolean transfer(Account from, Account to, long amount) {
            if (amount <= 0) return false;
            if (from.withdraw(amount)) {
                to.deposit(amount);
                return true;
            }
            return false;
        }

        public String getAccountNumber() { return accountNumber; }
        public String getOwnerName() { return ownerName; }
        public long getBalance() { return balance; }
    }

    public static void main(String[] args) {
        System.out.println("RUDRA MODI || 25DCS058");
        System.out.println();
        System.out.println("Practical 2 - Part B: MiniBank Account Generator & Invariant
Protection");
        System.out.println();
        Customer c1 = new Customer("Rudra", "rudra@bank.com", "9876543210");
        Customer c2 = new Customer("Aarav", "aarav@bank.com", "8765432109");

        Account[] accounts = new Account[3];
        accounts[0] = new Account(c1.getName(), 5000);
        accounts[1] = new Account(c2.getName(), 2000);
        accounts[2] = new Account("Joint Account", -100);

        System.out.println("Testing Transactions:");
        accounts[0].deposit(2000);
        accounts[0].withdraw(3000);
        accounts[0].withdraw(8000);
```

```
        Account.transfer(accounts[0], accounts[1], 1000);

        for (Account a : accounts) {
            System.out.println("Owner: " + a.getOwnerName() + " | AccNo: " +
a.getAccountNumber() + " | Balance: Rs. " + a.getBalance());
        }
    }
}
```

Output:

```
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals> java Presentation\Sem-3.OOP.Prac-2\PartB.java
RUDRA MODI || 25DCS058

Practical 2 - Part B: MiniBank Customer Profile & Encapsulated Accounts

Customer Created: [ID: CUST101 | Name: Rudra Modi | Phone: 9876543210]
Account AC1001 assigned with Initial Balance: Rs. 10000
Depositing Rs. 5000 -> Deposit Confirmed. New Balance: Rs. 15000
Withdrawing Rs. 3000 -> Withdrawal Successful. Remaining Balance: Rs. 12000
Withdrawing Rs. 20000 -> Withdrawal REJECTED (Insufficient funds)
PS E:\B.Tech CSE CHARUSAT\SEM 3\OOP\Practicals>
```

Key Questions & MiniBank Architecture Analysis:

Q1: How do private fields with public getters enforce encapsulation in MiniBank?

Answer: The balance and account details are declared private so external code cannot arbitrarily modify funds. Public getters provide read access, while transactions must execute through validated methods (deposit() and withdraw()) that verify amount positivity and sufficient funds.

Q2: Why is accountNumber declared final and how is it auto-generated?

Answer: An account number is a permanent immutable identifier that must never change post-creation. A private static long counter is incremented during instantiation to generate globally unique, sequential account numbers (e.g., 'AC1001').